02613 Python and High-Performance Computing

Re-exam 2024

Written examination date: 21.08.2024

Page 1 of 10 pages

Course name: Python and High-Performance Computing

Course number: 02613

Aids allowed: Written works of reference are permitted

Exam duration: 4 hours

Weighting: The grade is determined based on an overall assessment of all answers to the exam questions.

For all *non-multiple choice* problems, when answering the different questions, **remember to show/explain your calculations and motivate your answers**. If you make additional assumptions, remember to briefly explain them. Failure to do so may cause substantial reduction of the number of points given. You may write your answers directly on the exam set, on separate papers, or both. For answers on separate papers, remember to clearly indicate what question your are answering.

For all *multiple choice* problems you may simply mark your chosen answer with no additional justification. If you wish to change your answer, cross out your current mark and write your chosen option next to the question.

Unless otherwise specified:

- All arrays are NumPy arrays and stored row-wise.
- `np` refers to the NumPy Python module.

You are a performance consultant and have been sent by DTU to help out a company called Strategic Networked Advanced Intelligence Logistics. They are doing many different kinds of data processing, all of which are currently running too slow.

One of the engineers is using batch jobs to perform some data processing. They show you a job script they are working on:

```
#!/bin/bash
#BSUB -J classify
#BSUB -q hpc
#BSUB -W 02:00
#BSUB -R "rusage[mem=4GB]"
#BSUB -n 8
#BSUG -R "span[hosts=1]"
#BSUB -o proc_%J.out
#BSUB -e proc_%J.err

python classify.py data.csv
```

1. What resources does this job script request? Mention maximum wall-time, memory usage and cpu cores.

To improve performance, the engineers have restructured `classify.py` to run their algorithm on a GPU.

2. How would you change the job script above such that the program is run on a node with a GPU?

Another engineer is working on a different program using CPU parallel processing. The program takes 10 minutes to run on 4 processors. They have estimated the parallel fraction as $F = 0.8$.

3. How many minutes would the program take to run on 1 processor?

An engineer now claims they can reduce the run time of the non-parallelizable part by 3 minutes.

4. Given the reduced serial run time, what will the new run time on 4 processors be?

They now send you to another department. Here, an engineer is working on an application where they have a matrix of size $1\,000 \times 100\,000$ with data type `float64`. The matrix is too large for them to load it into memory, so they will store it as a Zarr array and read it in blocks. For their application, they need to compute the sum of a given set of columns, i.e., their code is:

```
def process(fname, columns):
    x = zarr.open(fname, mode='r')
    s = 0
    for i in columns:
        s += x[:, i].sum()
    return s
```

5. Only considering the above application, which of the following block shapes for the Zarr array will achieve the best performance?
   (a) $10 \times 10\,000$ (b) $100 \times 1\,000$ (c) $1\,000 \times 100$

6. How much memory will it take to load a single block into memory?

Satisfied with your help, they now send you to the simulation department. Here, they are trying to optimize a function named `simulate`. To this end, they have run it through a line profiler and gotten the following output:

```
Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     3                                           @profile
     4                                           def simulate(x, y, n_steps):
     5         1          1.0      1.0      0.0      z = 0.0
     6         1          1.0      1.0      0.0      n = len(x)
     7     10001       2000.0      0.2     13.3      for i in range(n_steps):
     8     10000       5000.0      0.5     33.3          a = x[i] * x[i] + 4
     9     10000       5000.0      0.5     33.3          b = y[n - i - 1] / x[i]
    10     10000       3000.0      0.3     20.0          z = z + a / b
    11         1          0.3      0.3      0.0      return z
```

7. How many steps was the simulation run for, i.e., what was the value of `n_steps`?

The time unit for the above profiler output is in microseconds (µs) and 1 s = 10^6 µs. Assume `x` and `y` have type `float32`.

8. How many FLOP/s (floating point operations per second) does the above function achieve?
   (a) $2.67 \cdot 10^6$ FLOP/s (b) $3.33 \cdot 10^6$ FLOP/s (c) $4.67 \cdot 10^6$ FLOP/s

Instead of using a Python loop, you suggest computing the answer using NumPy array operations.

9. Which of the following NumPy expressions will compute the same value as the `simulate` function in the above profiler output?
   (a) `(x * x + 4) / (y[::-1] / x)`
   (b) `np.sum((x * x + 4) / (y / x))`
   (c) `np.sum((x * x + 4) / (y[::-1] / x))`

Impressed with your knowledge about NumPy, another engineer asks for your help with a NumPy broadcasting question. They have two NumPy arrays `a` and `b`. The shape of `a` is $100 \times 1 \times 6 \times 3$ and the shape of `b` is $100 \times 1 \times 3$.

10. Given the operation `c = a + b`, what is the shape of the output array `c`?
    (a) $100 \times 100 \times 6 \times 3$ (b) $100 \times 1 \times 6 \times 3$ (c) Won't broadcast.

Next, you are sent to the CUDA department. Here, they are working on a CUDA kernel that takes an image, `x`, which is stored as an $\texttt{n} \times \texttt{n}$ array, as an input. It then computes a 3×3 average around every pixel and this result in then added to an output array `y`. Their code is:

```
@cuda.jit
def average3x3(x, y, n):
    row, col = cuda.grid(2)
    s = 0.0
    if 1 <= row < n - 1 and 1 <= col < n - 1:
        for j in range(-1, 2):
            for i in range(-1, 2):
                s += x[row + i, col + j]
        y[row, col] += s / 9  # Add to y
```

11. Which of the following thread block configurations will have the *best* performance, i.e., run the *fastest*?
    (a) 16×16 (b) 256×1 (c) 1×256

12. Explain your reasoning for the answer to the previous question:

The engineers now want to apply this kernel to several images and add up all the results. The images are stored in an `m` × `n` × `n` array `x_all`, where `m` is the number of images. Since only one `n` × `n` image can fit in GPU memory, they apply the `average3x3` kernel one image at a time using the following code:

```
def sumavg(x_all, threadsperblock, blockspergrid):
    y = np.zeros(x_all.shape[1:], dtype=x_all.dtype)
    for x in x_all:
        average3x3[blockspergrid, threadsperblock](x, y, len(x))
    return y
```

Assume `x_all` is a $100 \times 2\,000 \times 2\,000$ NumPy array in host memory.

13. How many host to device (HtoD) and device to host transfers (DtoH) will `sumavg` perform if called with `x_all`?

14. How many transfers would be needed in an optimal implementation of `sumavg`?

Happy with your analysis, an engineer from another department wants your input on a large processing task. They are processing ten large files using a job array after which a final job will compute the end result once the job array has finished successfully. They have created the following two job scripts for this purpose:

```
#!/bin/bash
#BSUB -J prepare[1-10]
#BSUB -q hpc
#BSUB -W 2:00
#BSUB -n 1
#BSUB -R "rusage[mem=512MB]"
#BSUB -o batch_output/prepare_%I_%J.out
#BSUB -e batch_output/prepare_%I_%J.err




python prepare.py ${LSB_JOBINDEX}
```

bjob_prepare.sh

```
#!/bin/bash
#BSUB -J compute
#BSUB -q hpc
#BSUB -W 5:00
#BSUB -n 8
#BSUB -R "span[hosts=1]"
#BSUB -R "rusage[mem=512MB]"
#BSUB -w done(prepare)
#BSUB -o batch_output/compute_%J.out
#BSUB -e batch_output/compute_%J.err


python compute.py
```

bjob_compute.sh

They submit the jobs and after a few hours they call `bjobs -A` to check the progress. They get the following output:

```
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
22185215    prepare[  user123      10    0    4    5    1     0     0     0
```

15. When will the `compute` job start?
    (a) As soon as possible
    (b) When the remaining jobs have finished
    (c) Never

Satisfied with your advice, the engineer sends you on to their final department. Here, another engineer is trying to sum all numbers in a large $n \times n$ matrix `x`. To speed this up, they want to use parallel processing. They get the idea to sum each row or column in parallel and then sum the row/column sums serially afterwards. However, they are unsure whether performing the parallel sum over the rows or columns is best. Concretely, they have the following two implementations:

```
def summatrix(x, n_cores):
    with ThreadPool(n_cores) as p:
        # Sum rows in parallel
        rowsums = p.map(
            lambda a: a.sum(), x
        )
    return sum(rowsums)
```

Parallel row sum implementation

```
def summatrix(x, n_cores):
    with ThreadPool(n_cores) as p:
        # Sum columns in parallel
        colsums = p.map(
            lambda a: a.sum(), x.T
        )
    return sum(colsums)
```

Parallel column sum implementation

Assume n is large enough that a row/column cannot fit in the CPU cache.

16. Which of the two parallel implementations would you expect to be faster? Explain your reasoning:

This uses multi-threading. The engineer worries whether it would be better to use multi-processing instead.

17. Is it appropriate to use multi-threading instead of multi-processing in this case? Explain why/why not:

Another engineer states that since you are summing all numbers, you should just reshape `x` to a flat array of length n^2 and then perform a parallel reduction. Assume you have unlimited computational resources available.

18. How much faster would a parallel reduction be compared to the approach from question 16? Explain your reasoning and provide your answer as a function of n:

End of exam